

~~EduPulse~~

Software Design Document
Version 1.0

Author: Rishabh

Date: May 2026

Status: Production

Repository: github.com/Rishabhworkspace/Edupulse

Table of Contents

- 1. Executive Summary
- 2. System Architecture
- 3. Technology Stack
- 4. Database Design
- 5. API Reference
- 6. Authentication & Security
- 7. Frontend Architecture
- 8. Feature Modules
- 9. CI/CD Pipeline
- 10. Future Roadmap

1. Executive Summary

EduPulse is a full-stack Learning Management System (LMS) built on the MERN stack (MongoDB, Express.js, React 18, Node.js). The platform serves three user roles - Students, Instructors, and Administrators - each with a dedicated dashboard and feature set.

The system enables course creation with rich multimedia curricula, real-time progress tracking with gamification (XP, streaks, badges), Stripe-powered payment processing, and a community Q&A layer. The architecture follows a monorepo pattern using npm workspaces.

1.1 Project Goals

- Provide a premium, interactive learning experience for students
- Empower instructors with a powerful Course Studio for curriculum authoring
- Give administrators full control over users, content, and revenue
- Ensure security through JWT auth, rate limiting, and input sanitization
- Maintain code quality via CI/CD with automated lint, test, and deploy stages

1.2 Scope

v1.0 covers: user auth (email + Google OAuth), course catalog with search/filters, immersive course player (video, articles, quizzes), enrollment and payment flow, student dashboard with analytics, instructor course management, admin panels, and contact/support system.

2. System Architecture

2.1 High-Level Overview

Client-server architecture. React client (port 5173) communicates with Express API server (port 5000) via proxied HTTP. Server interfaces with MongoDB Atlas and external services (Stripe, Cloudinary).

2.2 Monorepo Structure

Directory	Purpose	Key Technologies
client/	React SPA frontend	React 18, Vite, Redux Toolkit
server/	Express API backend	Express.js, Mongoose, Passport
package.json	Root workspace config	npm workspaces, concurrently
.github/	CI/CD workflows	GitHub Actions, Vercel

2.3 Request Flow

1. User interacts with React UI
2. Redux dispatches API call via Axios
3. Interceptor attaches JWT access token
4. Express middleware chain: Rate Limiter > Auth > Validator > Controller
5. Controller calls Service layer querying MongoDB via Mongoose
6. JSON response returned; Redux updates; UI re-renders
7. On 401, interceptor auto-refreshes token

3. Technology Stack

3.1 Frontend

Technology	Version	Purpose
React	18.x	Component-based UI framework
Vite	5.x	Dev server and production bundler
Redux Toolkit	2.x	Centralized state management
React Router	6.x	Client-side routing with lazy loading
Framer Motion	11.x	Declarative animations
Axios	1.x	HTTP client with interceptors
Lucide React	-	Modern icon library

3.2 Backend

Technology	Version	Purpose
Node.js	18+	JavaScript runtime
Express.js	4.x	REST API web framework
Mongoose	7.x	MongoDB ODM
Passport.js	0.7.x	Google OAuth middleware
Stripe SDK	14.x	Payment processing
Cloudinary	1.x	Image/video CDN
Winston	3.x	Structured logging
bcryptjs	-	Password hashing (12 rounds)

3.3 Security Libraries

- helmet - Secure HTTP headers
- express-rate-limit - DDoS protection
- express-mongo-sanitize - NoSQL injection prevention
- jsonwebtoken - JWT generation/verification
- cookie-parser - HttpOnly cookie handling

4. Database Design

4.1 Overview

MongoDB Atlas with 10 Mongoose collections. Optimized with compound indexes, text search, and .lean() queries.

4.2 User Collection

Field	Type	Details
name	String	Required, max 100
email	String	Unique, lowercase, indexed
password	String	bcrypt hashed, select:false
role	Enum	student instructor admin
isVerified/isBanned	Boolean	Account status
xpPoints/streak	Number	Gamification
badges	[Object]	Name, icon, awardedAt
wishlist	[ObjectId]	Ref: Course
oauth	[Object]	Provider + providerId
refreshToken	String	select: false

4.3 Course Collection

Field	Type	Details
title/slug	String	Required, auto-slug
price	Number	Min 0, default free
instructor	ObjectId	Ref: User
category/level	String	Filtering fields
status	Enum	draft review published archived
rating/enrolledCount	Number	Aggregated
outcomes/requirements	[String]	Learning objectives
isFeatured	Boolean	Admin-promoted

4.4 Supporting Collections

Collection	Key Fields	Relationships
Lesson	title, type, content, order	Belongs to Course
Enrollment	user, course, progress	User-Course M:N
Order	user, course, amount, stripeId	Payment record

Collection	Key Fields	Relationships
Discussion	course, user, replies, votes	Q&A threads
Review	user, course, rating, comment	Unique user+course
Coupon	code, type, discount, expiry	Applied to Orders
Category	name, slug, courseCount	Grouping
Contact	name, email, subject, message	Support tickets

5. API Reference

All endpoints under `/api/v1/`. Auth via Bearer tokens. Rate limiting applied globally.

5.1 Authentication

Method	Endpoint	Description	Auth
POST	<code>/auth/register</code>	Create account	No
POST	<code>/auth/login</code>	JWT token pair	No
POST	<code>/auth/refresh</code>	Refresh token	Cookie
POST	<code>/auth/logout</code>	Invalidate	Yes
POST	<code>/auth/forgot-password</code>	Reset email	No
PUT	<code>/auth/reset-password/:token</code>	Reset	No
GET	<code>/auth/google</code>	OAuth initiate	No
GET	<code>/auth/google/callback</code>	OAuth callback	No

5.2 Courses

Method	Endpoint	Description	Auth
GET	<code>/courses</code>	List + filters + pagination	No
GET	<code>/courses/:slug</code>	Full detail + curriculum	No
POST	<code>/courses</code>	Create course	Instructor
PUT	<code>/courses/:id</code>	Update metadata	Instructor
DELETE	<code>/courses/:id</code>	Archive	Admin
POST	<code>/courses/:id/enroll</code>	Enroll (free)	Student
PUT	<code>/courses/:id/progress</code>	Update progress	Student

5.3 Payments & Other

Method	Endpoint	Description	Auth
POST	<code>/payments/create-checkout</code>	Stripe session	Student
POST	<code>/payments/verify</code>	Verify + enroll	Student
POST	<code>/payments/webhook</code>	Stripe webhook	Stripe
GET/PUT	<code>/users/me</code>	Profile CRUD	Yes
CRUD	<code>/coupons</code>	Coupon mgmt	Admin
POST	<code>/contact</code>	Support ticket	No

6. Authentication & Security

6.1 JWT Token Architecture

Access Token: 15 min, Authorization header

Refresh Token: 7 days, HttpOnly secure cookie

On expiry, Axios interceptor auto-calls /auth/refresh for seamless UX.

6.2 Google OAuth 2.0

Passport.js GoogleStrategy. Creates/links user, generates JWT pair, redirects to /oauth-success.

6.3 Password Security

- bcryptjs hashing with 12 salt rounds
- Min 6 chars at model level
- Password excluded from queries (select: false)
- Reset via crypto-random tokens (1hr expiry)

6.4 Middleware Stack

Middleware	Purpose
helmet	Secure HTTP headers (CSP, HSTS)
express-rate-limit	DDoS protection
express-mongo-sanitize	NoSQL injection prevention
cors	Whitelist-based origin control
compression	Gzip compression

7. Frontend Architecture

7.1 Component Hierarchy

App (ErrorBoundary + Suspense)

MainLayout > Public + Auth pages

DashboardLayout > Student + Instructor + Admin pages

CoursePlayerPage (fullscreen immersive)

7.2 State Management

Redux Toolkit: authSlice (session, tokens) + courseSlice (catalog, filters). Async via createAsyncThunk.

7.3 Routing

- React Router v6 with nested layouts
- Lazy-loaded via React.lazy() + Suspense
- ProtectedRoute / GuestRoute / RoleRoute guards
- Chunk preloading on hover

7.4 Key Components

Component	Purpose
VideoPlayer	Responsive video playback
PlayerSidebar	Curriculum nav + progress
CourseCard	Catalog card with rating
NotesPanel	Per-lesson notes
QuizPlayer	Interactive quizzes
ErrorBoundary	Crash recovery
MockPaymentPopup	Dev payment testing

8. Feature Modules

8.1 Course Discovery

- Text search across title, description, tags
- Filter by category, level, price, rating
- Responsive grid with animated cards
- Detail page with hero, curriculum, reviews

8.2 Course Player

- Full-screen with collapsible sidebar
- Video, article, quiz lesson types
- Notes panel synced per lesson
- Progress tracking + 15s timeout protection

8.3 Student Dashboard

- Overview with enrolled count, XP, streak
- MyCourses, Schedule, Achievements, Assignments
- Saved courses and community forum

8.4 Instructor Tools

- Course Studio for creation + curriculum
- Revenue analytics + Q&A Dashboard

8.5 Admin Panel

- User management with ban/role controls
- Course moderation + order tracking
- Coupon system (flat/percentage)

8.6 Payment System

- Stripe Checkout + mock dev mode
- Coupon validation + auto-enrollment
- Webhook for async Stripe events

9. CI/CD Pipeline

Stage	Tool	Description
Checkout	actions/checkout	Clone repo
Setup	actions/setup-node	Node.js 18.x
Install	npm ci	Dependencies
Lint	ESLint	Static analysis
Test	Jest + Vitest	Server + client
Build	Vite	Production bundle
Deploy	Vercel CLI	Edge deployment

9.2 Environment Variables

Variable	Required By	Description
MONGO_URI	Server	MongoDB Atlas connection
JWT_ACCESS_SECRET	Server	Access token key
JWT_REFRESH_SECRET	Server	Refresh token key
STRIPE_SECRET_KEY	Server	Stripe API secret
CLOUDINARY_*	Server	Cloud config
VITE_API_URL	Client	Backend API URL

10. Future Roadmap

Phase	Feature	Description	Priority
1	Live Sessions	WebRTC video + chat	High
2	AI Tutor	AI Q&A + quiz gen	High
3	Mobile App	React Native + offline	Medium
4	Marketplace	Payouts + affiliates	Medium
5	Analytics V2	Cohort analysis	Low
6	Localization	i18n support	Low

This document provides a comprehensive overview of the EduPulse platform architecture, design decisions, and implementation details.

--- End of Document ---